Python Functions

Key Concepts:

- **Function Definition**: A function in Python is a block of reusable code that performs a specific task. It is defined using the def keyword, followed by the function name, parameters in parentheses, and a colon.
- **Components of a Function**:
  1. **Function Header**: Starts with def, includes the function name and parameters.
  2. **Function Body**: Contains the statements that execute the function's task, indented for structure.
  3. **Return Statement**: Optional; returns the output of the function.

Example:

```python
def my_adder(a, b, c):
    """Function to sum three numbers"""
    return a + b + c
```

Usage:

```python
result = my_adder(1, 2, 3)  # Returns 6
```

Best Practices:

- Use descriptive names for functions and variables.
- Comment extensively to clarify the function's purpose and logic.
- Regularly save your work.

Understanding Function Parameters

Input Arguments:

- Functions can accept multiple input arguments, which are variables provided by the user or another function.
- Example:

```python
def multiply(x, y):
    return x * y
result = multiply(4, 5)  # Returns 20
```

Output Parameters:

- Functions may return multiple values as a tuple.
- Example:

```python
def my_trig_sum(a, b):
    out1 = np.sin(a) + np.cos(b)
    out2 = np.sin(b) + np.cos(a)
    return out1, out2
c, d = my_trig_sum(2, 3)
```

Error Handling:

- Python functions assume input arguments will be of the expected type. If not, errors like TypeError may occur.
- Example of error:

```python
def my_adder(a, b, c):
    return a + b + c

result = my_adder(1, 2, "three")  # Raises TypeError
```

Advanced Function Features

Multiple Outputs:

- Functions can return multiple outputs, typically as a tuple, allowing the user to unpack the values into separate variables.
- Example

```python
def calculate(a, b):
    return a + b, a * b

sum_result, product_result = calculate(5, 10)
```

Default Parameters:

- Functions can have default parameter values, which are used if no argument is provided during the function call.
- Example:

```python
def greet(name="World"):
    print(f"Hello, {name}!")

greet()  # Prints "Hello, World!"
greet("Alice")  # Prints "Hello, Alice!"
```

Composition of Functions:

- Functions can be composed, meaning the output of one function can be used as the input for another.
- Example:

```python
def add(a, b):
    return a + b

def square(x):
    return x * x

result = square(add(2, 3))  # Returns 25
```

Understanding Local Variables in Python

- **Local Variables** are variables that are declared inside a function.
- These variables are **accessible only within that function** and have their own memory block separate from the rest of the program.
- The value of a local variable is not affected by or does not affect variables outside the function, even if they share the same name.

Example:

```python
def my_adder(a, b, c):
    out = a + b + c
    print(f'The value of out within the function is {out}')
    return out

out = 1
d = my_adder(1, 2, 3)
print(f'The value of out outside the function is {out}')
```

Output:

- Inside the function: out = 6
- Outside the function: out = 1

Key Points:

- When the function is executed, a **new memory block** is created.
- Variables inside the function do not interfere with variables outside it.

Understanding Global Variables in Python

- **Global Variables** are declared outside of functions and are accessible anywhere in the program.
- They can be read and modified both inside and outside functions, using the global keyword.

Common Issues:

- Without declaring a variable as global, attempts to modify it inside a function can lead to errors like UnboundLocalError.

Example Without Global Keyword:

```python
n = 42

def func():
    print(f'Within function: n is {n}')
    n = 3
    print(f'Within function: n is now {n}')

func()
```

Error: UnboundLocalError: local variable 'n' referenced before assignment

Solution Using Global Keyword:

```python
n = 42

def func():
    global n
    print(f'Within function: n is {n}')
    n = 3
    print(f'Within function: n is now {n}')

func()
print(f'Outside function: n is {n}')
```

Output:

- Inside function: n = 42, then n = 3
- Outside function: n = 3

Practical Implications and Best Practices

Why Separate Memory Blocks?

- **Efficiency**: Prevents functions from interfering with each other in large projects.
- **Modularity**: Allows independent development and debugging of functions without concern for variable name conflicts.

Best Practices:

1. **Minimize Global Variables**: Overuse can lead to complex dependencies and harder-to-debug code.
2. **Use Functions Effectively**: Leverage local variables within functions to encapsulate functionality and maintain clear scope boundaries.
3. **Global Keyword Use**: Use cautiously and only when necessary to modify global variables inside functions.

Nested Functions in Python

- **Definition**:
  - A nested function is a function defined inside another function (the parent function).
  - The nested function can only be called by the parent function.

- **Why use nested functions?**:
  - Useful when a task needs to be performed multiple times within the parent function but is not required outside.
  - Helps in organizing the code and preventing redundancy.

- **Example**:

```python
def parent_function():
    def nested_function():
        # Code for nested function
    # Code that calls the nested function
```

Memory and Scope in Nested Functions

- **Memory block**:
  - A nested function has its own separate memory block, independent of the parent function.
  - This allows variables with the same name to exist in both the parent and nested functions without conflict.

- **Practical Example**:

```python
def my_dist_xyz(x, y, z):
    def my_dist(x, y):
        return np.sqrt((x[0]-y[0])**2 + (x[1]-y[1])**2)
    d0 = my_dist(x, y)
    d1 = my_dist(x, z)
    d2 = my_dist(y, z)
    return [d0, d1, d2]
```

- Here, my_dist() is a nested function that computes distances between 2D points.

Advantages of Using Nested Functions

- **Code clarity**:
  - Organizing complex functions into smaller, task-specific sub-functions reduces clutter and enhances readability.

- **Error reduction**:
  - Nested functions reduce the risk of mistakes (like formula errors), as computations are centralized within a single subfunction.

- **Cleaner code**:
  - Without a nested function, the parent function may become cluttered with repeated logic, increasing the chances of errors and making it harder to maintain.

Lambda Functions in Python

- **Definition**: A *lambda function* is an anonymous function, meaning it has no name and is typically used for short, simple operations.

- **Syntax**: Lambda functions are written using the lambda keyword followed by any number of arguments, and an expression to be evaluated.

```python
lambda arguments: expression
```

- **Use Case**: These functions are commonly used when a simple function is needed for a single task, especially in scenarios such as sorting, filtering, or applying functions in a concise manner.

Example:

- To square a number:

```python
square = lambda x: x**2
print(square(2))  # Output: 4
```

Practical Applications of Lambda Functions

- **Lambda with Sorting**: Lambda functions are often used in sorting custom objects. For example, sorting a list of tuples based on the second element:

```python
sorted([(1, 2), (2, 0), (4, 1)], key=lambda x: x[1])
# Output: [(2, 0), (4, 1), (1, 2)]
```

Lambda for Simple Math Operations: Lambda functions allow quick mathematical operations without the need to define full functions.

```python
my_adder = lambda x, y: x + y
print(my_adder(2, 4))  # Output: 6
```

Advantages:

- Compact code.
- Useful in functions like map(), filter(), and sorted() where a small function is required.

Limitations and Best Practices

- **Limitations**:
  - Lambda functions are limited to one expression and are not suitable for complex operations.
  - Lack of a name makes debugging harder for more complicated functions.

- **Best Practices**:
  - Use lambda functions for simple tasks such as mathematical operations, data sorting, or filtering.
  - Avoid using them for large or complex functions where traditional function definitions are more appropriate.

Functions as Arguments

- **Key Points**:
  - In Python, functions are treated as first-class objects, meaning they can be passed as arguments to other functions.
  - This capability allows for greater flexibility in coding, making functions interchangeable.

  - **Example**: The max function can be assigned to a variable f, and using f is equivalent to calling the max function.

```python
f = max
print(f([2, 3, 5]))  # Output: 5
```

Custom Function with Function Argument

- **Key Points**:
  - You can define custom functions that accept other functions as arguments.
  - **Example**: A function my_fun_plus_one takes a function f and a value x as input, applies f to x, and adds 1 to the result

```python
import numpy as np

def my_fun_plus_one(f, x):
    return f(x) + 1

print(my_fun_plus_one(np.sin, np.pi/2))  # Output: 2.0
print(my_fun_plus_one(np.sqrt, 25))  # Output: 6.0
```

- **Concept**: This enables the combination of various mathematical operations in a streamlined way.

.

Lambda Functions as Input

- **Key Points**:
  - Lambda functions, which are small anonymous functions, can also be passed as arguments to other functions.

  - **Example**: Passing a lambda function to my_fun_plus_one for simple mathematical operations.

```python
print(my_fun_plus_one(lambda x: x + 2, 2))  # Output: 5
```

- **Advantage**: This reduces the need to define separate named functions for small operations, enhancing code efficiency and readability.